



INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA
LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO TÉCNICO

Sistema de Gestão de Clínica Médica

Relatório apresentado ao Instituto Politécnico da Universidade Kimpa Vita, como requisito para obtenção da nota de exame, na cadeira de Linguagem de Programação IV.

Orientado por: Moyo Kanivengidio

LISTA DE INTEGRANTES DA EQUIPA:

- 1-Levi Manuel Alberto
- 2-Gonçalves Pinto Domingos
- 3-Matondo Alberto Maurício
- 4-Ramos Brito Fernando
- 5-António Neves José

1-Introdução

O presente relatório técnico descreve o processo de desenvolvimento do Sistema de Gestão de Clínica Médica, trabalho prático realizado no âmbito da disciplina de Linguagem de Programação VI, do 2º Ano, sob orientação do Docente Moyo Kanivengidio.

O sistema foi desenvolvido com o objectivo de automatizar e otimizar os processos administrativos e clínicos de uma clínica médica, substituindo processos manuais e fragmentados por uma solução integrada, eficiente e segura.

A solução implementada permite a gestão de pacientes, médicos, consultas e prontuários eletrónicos, com controlo de acesso baseado em perfis de utilizador (Administrador, Médico e Rececionista), além de geração de relatórios gerenciais.

1.1-Objectivos

Gerais:

Construir uma aplicação desktop completa em Java, aplicando as melhores práticas de engenharia de software.

Específicos:

- Aplicar os pilares da Programação Orientada a Objetos (POO): encapsulamento, herança, polimorfismo e abstração;
- Desenvolver interfaces gráficas intuitivas utilizando Java Swing;
- Implementar persistência de dados com MySQL via JDBC;
- Aplicar os padrões arquiteturais MVC (Model-View-Controller) e DAO (Data Access Object);
- Implementar controlo de acesso com perfis de utilizador e autenticação segura com hash SHA-256.

2. ANÁLISE DE REQUISITOS

2.1-Requisitos Funcionais

Os requisitos funcionais definem as funcionalidades que o sistema deve oferecer aos seus utilizadores. A tabela seguinte resume os módulos implementados:

Módulo	Funcionalidades Implementadas
Gestão de Pacientes	Cadastro, edição, inativação e pesquisa de pacientes por nome ou CPF. Registo de histórico médico.
Gestão de Médicos	Cadastro, edição, inativação e pesquisa de médicos. Registo de especialidade e CRM.
Gestão de Consultas	Agendamento, edição, cancelamento e controlo de status (Agendada, Confirmada, Realizada, Cancelada). Verificação automática de conflito de horários.
Prontuários Eletrónicos	Criação e consulta de prontuários vinculados a consultas realizadas. Registo de diagnóstico, prescrição e observações.
Controlo de Acesso	Sistema de login com perfis (Admin, Médico, Rececionista). Autenticação com hash SHA-256. Restrição de funcionalidades por perfil.
Relatórios Gerenciais	Relatório de consultas por médico e período, listagem de pacientes ativos e resumo de consultas por status.

2.2 Requisitos Não Funcionais

- Usabilidade: interface intuitiva com navegação clara e mensagens de validação informativas;
- Segurança: senhas armazenadas com hash SHA-256, uso de PreparedStatement para prevenir SQL Injection;
- Desempenho: respostas rápidas nas consultas ao banco de dados com uso de índices;
- Manutenibilidade: código modular com alta coesão e baixo acoplamento;
- Portabilidade: desenvolvido em Java, compatível com Windows, Linux e macOS.

3-ARQUITETURA DO SISTEMA

3.1 Padrão MVC (Model-View-Controller)

O sistema foi desenvolvido seguindo rigorosamente o padrão arquitetura MVC, que separa a aplicação em três camadas lógicas distintas, promovendo a separação de responsabilidades e facilitando a manutenção e evolução do código.

Camada	Responsabilidade	Classes Principais
Model	Representa as entidades do domínio e contém as regras de negócio.	Usuário, Paciente, Medico, Consulta, Prontuário
View	Responsável pela apresentação dos dados e captura das interações do utilizador.	TelaLogin, TelaPrincipal, TelaPacientes, TelaConsultas, TelaProntuarios, TelaRelatorios
Controller	Intermediário entre View e Model. Processa as requisições e actualiza a View.	UsuarioController, PacienteController, MedicoController, ConsultaController, ProntuarioController

3.2 Padrão DAO (Data Access Object)

O padrão DAO foi implementado para isolar toda a lógica de acesso ao banco de dados das classes de negócio. Para cada entidade do domínio existe uma classe DAO correspondente que concentra todo o código JDBC.

- DAOGenerico: interface genérica que define as operações CRUD básicas (inserir, atualizar, deletar, buscar, listar Todos);
- PacienteDAO: implementa o CRUD completo para pacientes, incluindo pesquisa por nome e CPF;
- MedicoDAO: implementa o CRUD para médicos com JOIN na tabela de especialidades;
- ConsultaDAO: implementa agendamento com verificação de conflito de horários;
- ProntuarioDAO: implementa o CRUD para prontuários com JOIN em múltiplas tabelas;
- UsuarioDAO: implementa autenticação com verificação de hash SHA-256.

3.3 Estrutura de Pacotes

O projecto está organizado nos seguintes pacotes:

Pacote	Conteúdo
Com/clinica/model	Classes de domínio: Usuário, Paciente, Medico, Consulta, Prontuário
Com/clinica/dao	Classes de acesso a dados: DAOGenerico, PacienteDAO, MedicoDAO, ConsultaDAO, ProntuarioDAO, UsuarioDAO
Com/clinica/controller	Controllers: PacienteController, MedicoController, ConsultaController, ProntuarioController, UsuarioController
Com/clinica/view/forms	Telas Swing: TelaLogin, TelaPrincipal, TelaPacientes, TelaMedicos, TelaConsultas, TelaProntuarios, TelaRelatorios e formulários
Com/clinica/connection	ConexaoDB: gestão da conexão com MySQL (padrão Singleton)
Com/clinica/util	SenhaUtil: utilitário para hash SHA-256 de senhas

4.MODELO DE DADOS

4.1 Diagrama Entidade-Relacionamento (MER)

O banco de dados foi modelado com 8 tabelas relacionadas, implementado em MySQL. A seguir são descritas as principais entidades e os seus relacionamentos:

Tabela	Chave Primária	Relacionamentos
Usuários	Id (AUTO_INCREMENT)	Referenciado por medicos e pacientes
Especialidades	Id (AUTO_INCREMENT)	Referenciada por medicos (N:1)
Médicos	Id (AUTO_INCREMENT)	FK: usuario_id → usuarios, especialidade_id → especialidades
Pacientes	Id (AUTO_INCREMENT)	FK: cadastrado_por → usuarios
Consultas	Id (AUTO_INCREMENT)	FK: paciente_id → pacientes, medico_id → medicos, agendado_por → usuarios. UNIQUE (medico_id, data_hora)
Prontuários	Id (AUTO_INCREMENT)	FK: consulta id → consultas, medico_id → medicos
Medicamentos	Id (AUTO_INCREMENT)	Referenciada por prontuario_medicamentos
Prontuário_medicamentos	Id (AUTO_INCREMENT)	FK: prontuario_id → prontuarios, medicamento_id → medicamentos

4.2 Decisões de Modelagem

- A restrição UNIQUE (medico_id, data_hora) na tabela consultas impede conflito de horários directamente no banco de dados, garantindo integridade mesmo em acesso concorrente;
- O campo ativo (TINYINT) implementa exclusão lógica em vez de física, preservando o histórico de dados;
- As senhas são armazenadas como hash SHA-256, nunca em texto puro;

- O uso de PreparedStatement em todas as queries previne ataques de SQL Injection;
- Índices foram criados nos campos mais consultados (nome, cpf,.crm, data_hora) para otimizar o desempenho.

5. TECNOLOGIAS E FERRAMENTAS

Tecnologia	Versão	Finalidade
Java (JDK)	21 (Oracle OpenJDK)	Linguagem de programação principal
Java Swing	Incluída no JDK	Desenvolvimento das interfaces gráficas (GUI)
MySQL	8.x	Sistema de gestão de banco de dados relacional
JDBC	API Java	Conectividade entre Java e MySQL
MySQL Connector	9.7.0	Driver JDBC para MySQL
IntelliJ IDEA	2026.x	Ambiente de desenvolvimento integrado (IDE)
GitHub	-	Controlo de versão e repositório do código fonte

5.1 Justificativa das Escolhas

A escolha do Java Swing em detrimento do JavaFX deve-se à sua maturidade, estabilidade e por não requerer configurações adicionais para o ambiente de desenvolvimento utilizado, adequando-se melhor ao prazo disponível para o projecto.

O uso direto de JDBC em vez de frameworks ORM como o Hibernate foi uma decisão pedagógica consciente, permitindo aos membros da equipa compreender profundamente os mecanismos de acesso a dados e a linguagem SQL, conforme orientação que nos foi dada.

6-ASPECTOS DE IMPLEMENTAÇÃO

6.1 Controle de Acesso e Segurança

O sistema implementa autenticação baseada em perfis com três níveis de acesso:

- Administrador: acesso total a todos os módulos do sistema;
- Recepcionista: acesso à gestão de pacientes, agendamento de consultas e relatórios;
- Médico: acesso restrito a consultas e prontuários.

A segurança das senhas é garantida pelo algoritmo SHA-256, implementado na classe SenhaUtil. As senhas nunca são armazenadas em texto puro no banco de dados, apenas o seu hash hexadecimal de 64 caracteres é persistido.

6.2 Gestão de Conflitos de Horário

O sistema implementa verificação de conflito de horários em dois níveis:

- Nível 1: no código Java, através do método existeConflito do ConsultaDAO, que executa uma query COUNT antes de inserir uma nova consulta;
- Nível 2: no banco de dados, através da restrição UNIQUE (medico_id, data_hora) na tabela consultas, garantindo integridade mesmo em cenários de acesso concorrente.

CONCLUSÃO

O Sistema de Gestão de Clínica Médica foi desenvolvido com sucesso, cumprindo todos os requisitos funcionais e não funcionais estabelecidos. A aplicação demonstra a aplicação prática dos conceitos teóricos da disciplina, nomeadamente a Programação Orientada a Objetos, os padrões arquiteturais MVC e DAO, a integração com banco de dados via JDBC e o desenvolvimento de interfaces gráficas com Java Swing.

As principais dificuldades encontradas durante o desenvolvimento relacionaram-se com a configuração do ambiente de desenvolvimento e a integração entre as diferentes camadas da arquitetura. Estas dificuldades foram superadas através do estudo aprofundado da documentação e da aplicação sistemática dos padrões de projecto estudados.

O projecto contribuiu significativamente para o desenvolvimento das competências técnicas da equipa, consolidando conhecimentos em Java, SQL, arquitetura de software e boas práticas de engenharia de software.